

Сравнение инструментов управления секретами в Kubernetes

Домашнее задание 10

Содержание

1. Цель работы	3
2. Критерии сравнения	3
3. Описание инструментов	4
3.1. Встроенный механизм Kubernetes	4
3.2. HashiCorp Vault	5
3.3. External Secrets Operator	5
3.4. Sealed Secrets	6
3.5. SOPS	6
4. Сравнительная таблица	7
5. Рекомендации по выбору	8
6. Выводы	9

1. Цель работы

Цель — сравнить инструменты управления секретами в Kubernetes и определить, какой из них лучше подходит для разных рабочих ситуаций. Секрет в контексте этой работы — любые данные, которые нельзя хранить в открытом виде: пароли к базам данных, ключи API, сертификаты.

Сравниваются пять подходов:

- **Встроенный механизм Kubernetes** — стандартный объект Secret, доступный в любом кластере без дополнительной установки.
- **HashiCorp Vault** — специализированная система корпоративного уровня с полным набором функций безопасности.
- **External Secrets Operator (ESO)** — оператор, который синхронизирует секреты из внешних облачных хранилищ в кластер.
- **Sealed Secrets (Bitnami)** — инструмент для хранения зашифрованных секретов прямо в Git-репозитории.
- **SOPS (Mozilla)** — шифровщик файлов конфигурации с поддержкой разных источников ключей.

2. Критерии сравнения

Для оценки инструментов выбраны десять критериев. Каждый из них отвечает на конкретный практический вопрос, который возникает при работе с секретами в реальном проекте.

N	Вопрос, на который отвечает критерий	Почему это важно	Вес
1	Надёжно ли хранятся данные на сервере?	Если база данных кластера попадёт в чужие руки, злоумышленник не должен прочитать пароли как обычный текст.	20%
2	Система сама меняет пароли, не требуя ручной работы?	Чем дольше живёт один и тот же пароль, тем выше риск утечки. Автоматическая замена снижает этот риск.	15%
3	Конфигурации можно хранить в Git, не опасаясь утечки?	Команды хранят всю инфраструктуру в Git. Если туда случайно попадёт пароль, это станет проблемой.	15%
4	Инструмент создаёт одноразовые временные пароли?	Временный пароль, который истёк сам по себе, не нужно отзывать вручную. Это лучшая защита от утечек.	10%

5	Каждое обращение к секрету записывается в журнал?	При проверке нужно точно знать: кто, когда и какой секрет читал. Без журнала это невозможно.	15%
6	Инструмент хорошо встроен в Kubernetes?	Чем меньше дополнительного кода нужно писать, чтобы приложение получило свой пароль, тем лучше.	10%
7	Работает в разных облаках и на собственных серверах?	Зависимость от одного облака создаёт проблемы при миграции. Хороший инструмент работает везде.	5%
8	Легко установить и поддерживать?	Чем сложнее инструмент, тем больше времени уходит на его обслуживание вместо разработки продукта.	5%
9	Есть ли план на случай потери ключей или сбоя?	Если мастер-ключ потерян, а резервной копии нет — все секреты становятся недоступными навсегда.	5%
10	Сколько это стоит?	Некоторые инструменты бесплатны, другие требуют дорогостоящих лицензий при масштабировании.	—

Таблица 1. Критерии сравнения и их веса

Веса отражают приоритеты типичного production-проекта с требованиями к безопасности: на первом месте стоят защита данных, возможность аудита и безопасная работа с Git. Стоимость вынесена отдельно и не входит в итоговую оценку, но рассматривается при описании каждого инструмента.

3. Описание инструментов

3.1. Встроенный механизм Kubernetes

Kubernetes умеет хранить секреты с самого начала. Достаточно создать объект типа Secret, и Kubernetes положит его во внутреннюю базу данных. Приложение затем либо читает его через переменные окружения, либо находит нужные данные в файле, который Kubernetes автоматически подкладывает внутри контейнера.

Звучит удобно, но здесь есть серьёзная проблема: по умолчанию данные в базе данных кластера хранятся не зашифрованными. Они кодируются в формат base64, что легко обратимо и совсем не является защитой. Любой, кто получит прямой доступ к базе кластера, прочитает все пароли как обычный текст. Шифрование можно включить дополнительно, но оно не идёт из коробки и требует настройки.

Помимо этого, объект Secret нельзя безопасно добавить в Git-репозиторий: пароли там хранятся в открытом виде. Автоматической замены паролей нет, журнал обращений нужно настраивать вручную и отдельно.

Встроенный механизм хорошо подходит как основа, на которую опираются другие инструменты: ESO и Sealed Secrets в итоге тоже создают обычные объекты Kubernetes, которые приложение читает стандартным способом. Самостоятельно же встроенный механизм применим только для разработки или очень простых сценариев с тщательно настроенными правами доступа.

3.2. HashiCorp Vault

Vault — это специализированная система для хранения секретов корпоративного уровня. Данные хранятся зашифрованными, и каждое обращение к секрету записывается в подробный журнал: кто обратился, в какое время, к какому именно секрету, что получил в ответ. Такой уровень детализации необходим при работе с регуляторными требованиями.

Главная уникальная возможность Vault — это временные пароли. Vault умеет по запросу создать пароль к базе данных, который будет действовать ровно один час, а потом автоматически перестанет работать. Никто не хранит один постоянный пароль, который мог бы утечь или стать известен слишком широкому кругу людей. Такая же схема работает для доступа к облачным ресурсам и для выпуска сертификатов.

Обратная сторона: Vault сам по себе является сложным сервисом. Для надёжной работы нужен кластер из трёх серверов, регулярное резервное копирование, процедура восстановления при сбое и постоянный мониторинг. Если Vault окажется недоступен, все приложения, которые обращались к нему за паролями, не смогут запуститься. Это делает Vault мощным, но дорогим в обслуживании инструментом.

С версии 1.14 Vault перешёл на лицензию, которая ограничивает коммерческое использование. Community-версия остаётся бесплатной, но часть функций требует платной подписки. Vault оправдывает себя в командах от пяти-семи инженеров и выше, особенно если есть требования к аудиту.

3.3. External Secrets Operator

External Secrets Operator — это посредник между Kubernetes и внешними хранилищами секретов. Он не хранит секреты сам, а соединяет кластер с тем хранилищем, которое уже используется в организации: AWS Secrets Manager, Google Cloud Secret Manager, Azure Key Vault или тем же Vault.

Работает это так: разработчик описывает в Git-репозитории, какие секреты нужны приложению и где их взять. Само значение пароля при этом лежит в облаке. Оператор проверяет облако по расписанию и, если там что-то изменилось, сразу обновляет данные в кластере. Обновить пароль в облаке — значит автоматически обновить его во всех кластерах, которые подписаны на этот секрет.

Такой подход хорошо вписывается в работу с Git: в репозитории хранится только описание того, что нужно взять, но не сами значения паролей. Это безопасно.

Один нюанс: итоговый объект Secret, который оператор создаёт в кластере, всё равно попадает в базу данных Kubernetes. Если шифрование базы не включено, этот секрет хранится незащищённым так же, как при использовании встроенного механизма. Оператор решает задачу безопасного хранения и синхронизации, но не отменяет необходимость правильно настроить сам кластер.

3.4. Sealed Secrets

Sealed Secrets — это инструмент, который позволяет хранить зашифрованные секреты прямо в Git-репозитории. Разработчик берёт обычный пароль, шифрует его с помощью публичного ключа своего кластера командой `kubeseal`, и получает файл с зашифрованными данными. Этот файл можно безопасно добавить в репозиторий.

Расшифровать файл может только тот кластер, чей ключ был использован при шифровании. Даже если репозиторий открыт для всех желающих, никто посторонний не сможет прочесть пароли.

Sealed Secrets прост в использовании и не требует никакой внешней инфраструктуры. Он отлично подходит для небольших команд, которые работают с Git и не хотят тратить время на обслуживание сложного сервиса.

Есть два ограничения, о которых нужно знать заранее. Во-первых, секреты привязаны к конкретному кластеру: при переносе приложения в другой кластер придётся перешифровать все секреты с новым ключом. Во-вторых, резервная копия ключей кластера критически важна: если ключ потерян, все зашифрованные секреты станут нечитаемыми навсегда.

3.5. SOPS

SOPS — это инструмент командной строки от Mozilla для шифрования файлов конфигурации. Он понимает форматы YAML и JSON и шифрует не весь файл

целиком, а только значения внутри него. Благодаря этому в Git-репозитории хорошо видно, какие ключи конфигурации существуют, а их значения скрыты.

SOPS поддерживает разные источники ключей: облачные сервисы AWS, Google Cloud, Azure, а также автономный инструмент age, который работает полностью без интернета.

Интересная особенность — схема с несколькими ключами. Можно настроить так, что для расшифровки потребуются одновременно ключ из облака и локальный ключ разработчика. Это защищает от ситуации, когда один человек может расшифровать всё в одиночку.

Из практических ограничений: SOPS не является оператором Kubernetes и не следит за секретами автоматически. Чтобы изменение зашифрованного файла попало в кластер, нужно либо использовать Flux (который умеет читать SOPS-файлы напрямую), либо выполнять дополнительные шаги вручную. Ротация паролей не автоматизирована.

SOPS хорошо подходит командам, которые используют Flux для деплоя и хотят хранить конфигурацию рядом с обычными файлами инфраструктуры — зашифрованно, но в том же репозитории.

4. Сравнительная таблица

В таблице ниже каждый инструмент оценён по всем десяти критериям. Оценки даны по пятибалльной шкале: 5 — полностью удовлетворяет, 1 — практически не соответствует. В последней строке приведена итоговая оценка с учётом весов каждого критерия.

Критерий	Встроен. K8s	Vault	ESO	Sealed Secrets	SOPS
Данные защищены в базе кластера (20%)	2	5	3	4	4
Пароли меняются сами, без ручной работы (15%)	1	5	4	2	2
Конфиги безопасны в Git (15%)	1	3	5	5	4
Создаёт одноразовые временные пароли (10%)	1	5	3	1	1
Все обращения записываются в журнал (15%)	2	5	3	2	2
Хорошо встроен в Kubernetes (10%)	5	5	5	4	2

Работает в разных облаках и на своих серверах (5%)	4	5	5	2	4
Легко установить и поддерживать (5%)	5	1	4	5	3
Есть резервное копирование и план на сбой (5%)	2	3	5	2	3
Стоимость	Бесплатно	Бесплатно или платно	Бесплатно	Бесплатно	Бесплатно
Итоговая оценка	2.1 / 5	4.3 / 5	3.9 / 5	3.2 / 5	2.7 / 5

Таблица 2. Сравнение инструментов по взвешенной пятибалльной шкале

Несколько выводов из таблицы.

Vault занимает первое место по итоговой оценке, и это честно отражает его богатство возможностей. Но высокая оценка не означает, что он подойдёт всем: за ней стоит высокая сложность и потребность в выделенных людях для обслуживания.

ESO занимает второе место и при этом значительно проще в эксплуатации. Если проект уже использует облачное хранилище секретов, ESO добавляет очень мало лишней работы.

Sealed Secrets и SOPS получили средние оценки не потому, что плохие инструменты, а потому что не предназначены для всех сценариев. В своей нише — небольшие команды, GitOps, простота — они работают отлично.

Встроенный механизм Kubernetes набирает мало баллов как самостоятельное решение, но остаётся основой, на которую опираются все остальные инструменты.

5. Рекомендации по выбору

Правильный выбор зависит не от рейтинга в таблице, а от конкретной ситуации. Ниже описаны четыре типичных сценария.

Ситуация	Инструмент	Почему именно он
Небольшая команда, один кластер, нет строгих требований к аудиту	Sealed Secrets	Прост в использовании, не требует отдельной инфраструктуры, секреты безопасны в Git. Разработчик освоит инструмент за полчаса.
Большая команда, требования к аудиту, нужны временные пароли к базам данных	HashiCorp Vault	Единственный инструмент с полноценными временными паролями и подробным журналом, который удовлетворяет требованиям проверяющих органов.

Проект в AWS, GCP или Azure, облачное хранилище секретов уже используется	External Secrets Operator	Не нужна новая инфраструктура. Оператор сам подхватывает обновления из облака и распространяет их по всем кластерам.
Команда работает с Flux, хочет хранить всё в одном репозитории, нет облачной зависимости	SOPS с ключами age	Работает полностью автономно, без облака. Flux умеет читать SOPS-файлы напрямую. Один репозиторий — для всей инфраструктуры.

Таблица 3. Рекомендации по выбору инструмента в зависимости от ситуации

Важно понимать, что эти инструменты не заменяют друг друга, а часто используются вместе. Например, ESO берёт секреты из AWS Secrets Manager и помещает их в стандартные объекты Kubernetes, которые приложение читает обычным способом. Vault можно использовать как хранилище для ESO, получая тем самым и удобный GitOps-подход, и корпоративный уровень безопасности.

6. Выводы

По результатам сравнения можно сформулировать несколько практических выводов.

Встроенный механизм Kubernetes не является достаточным решением для производственного окружения в чистом виде. Данные в базе кластера по умолчанию не зашифрованы, секреты нельзя безопасно хранить в Git, автоматической замены паролей нет. Он хорошо работает как нижний слой в связке с другими инструментами, но не как единственное решение.

HashiCorp Vault предлагает лучший набор функций, но его стоит выбирать осознанно. Если в команде нет людей, которые могут и хотят его обслуживать, операционные затраты перевесят преимущества. Vault — это не просто инструмент, а отдельный сервис, который нужно поддерживать в рабочем состоянии постоянно.

External Secrets Operator — наиболее сбалансированный выбор для проектов в облаке. Он не требует отдельной инфраструктуры, хорошо вписывается в Git-подход и автоматически синхронизирует изменения. При этом его можно освоить значительно быстрее, чем Vault.

Sealed Secrets незаслуженно недооценён. Для большинства команд среднего размера это оптимальное решение: минимальные затраты на обслуживание, безопасное хранение в Git, простая работа с ArgoCD и Flux. Главное — не забывать делать резервные копии ключей кластера.

SOPS занимает свою нишу и хорошо её закрывает. Он полезен не как основной инструмент управления секретами приложений, а как способ зашифровать конфигурационные файлы инфраструктуры рядом с обычными манифестами. В связке с Flux это работает без лишних усилий.

Итоговый совет прост: не нужно выбирать самый мощный инструмент из возможных. Нужно выбирать тот, который команда сможет правильно настроить, понять и поддерживать. Неправильно настроенный Vault опаснее, чем хорошо настроенный Sealed Secrets.